MCCI BUSINESS SCHOOL

BC05

Traitement de flux en temps réel (Kafka, Flink)

Chapitre 2 - Présentation de Kafka

TP fil rouge

Déployer un cluster Kafka de A à Z : la chaîne du froid

Module	RNCP39765BC05FM04
Enseignant	RAKOTONIAINA HERIFANIRY Moise
Année	2026-2027
Version	V2 du 21/10/2026

Votre mission

Groupes de 2 à 4 **Durée estimée** 12 h 40 par étudiant : 9 h 05 de phases, 3 h 35 de livrables **Cours** sections 2.1 à 2.5 **Kafka** 4.3.1, KRaft à rôles séparés

Une entreprise fictive, FROIDIS, exploite un réseau de chambres froides industrielles. Elle veut détecter vite une dérive de température, suivre les mesures en temps réel et conserver les événements pour analyser les incidents. Aujourd'hui, les mesures arrivent dans un fichier partagé, étudié le lendemain : ce traitement par lots empêche de réagir dès l'apparition d'une anomalie.

Votre équipe conçoit, déploie et démontre le cluster Kafka qui servira de colonne vertébrale à la plateforme d'événements : réception et conservation des mesures, diffusion à plusieurs services, reprise après panne, administration. Le calcul des alertes sera confié à Flink au chapitre 3.

Besoins du client

Critère	Hypothèse de travail
Capteurs	500 capteurs de température
Fréquence moyenne	1 mesure par seconde et par capteur
Pic de charge	deux fois le débit moyen
Taille moyenne d'un message	200 octets, clé et valeur comprises
Ordre nécessaire	mesures ordonnées par capteur, pas à l'échelle globale
Historique brut	7 jours, pour rejouer les mesures
Historique des alertes	30 jours
Dernier état de chaque capteur	garder la dernière valeur de chaque clé
Garantie demandée	aucune perte de message confirmé, dans la limite des pannes prévues
Applications aval	tableau de bord, service d'alertes, analyse différée
Anomalie métier (exemple)	température supérieure à 8 °C pendant 2 minutes
Zones de production	trois zones indépendantes, un broker par zone

Ces chiffres sont des hypothèses de travail, pas des mesures d'un exploitant réel. Le seuil de 8 °C est une hypothèse pédagogique, pas une prescription sanitaire. C'est à vous de traduire ces besoins en paramètres Kafka.

Ce que votre groupe devra démontrer

N°	Critère de recette	Phase
1	Trois contrôleurs KRaft et un quorum de métadonnées sain	C
2	Trois brokers connectés au cluster, avec les identifiants 4, 5 et 6	C
3	Des topics métier répliqués, avec une politique de durabilité justifiée	A, D
4	L'écriture de mesures avec clé et leur lecture par deux groupes distincts	E, F
5	La sortie puis le retour d'une réplique dans l'ISR après une panne	G
6	Le refus des écritures quand `min.insync.replicas=2` n'est plus satisfait	G
7	Une démonstration de rétention, de compaction et de réaffectation de répliques	H, I
8	Un plan de supervision, de sécurité et de remise en service argumenté	J

Ce que vous avez déjà fait

Ce TP réunit, sur un cluster que vous construisez vous-mêmes, ce que vous avez pratiqué séparément : le quorum (exercice 1, phase C), les topics et les offsets (exercice 2, phases D et E), les segments (exercice 3, phase H), l'ISR (exercice 4, phase G), la compaction (exercice 5, phase H), la réaffectation (exercice 6, phase I), le choix des réglages selon le besoin (atelier de la séance 5, phases A et D) et le test de panne (séance 6, phase G).

Avant de commencer

À lire en entier avant la phase C

Poste de travail

- Docker Desktop (Windows, macOS) ou Docker Engine avec Compose (Linux), un terminal et un éditeur de texte.
- L'image `apache/kafka:4.3.1`, déjà utilisée pour les exercices. Si elle manque, Docker la télécharge : environ 240 Mo, 686 Mo une fois installée.
- De la mémoire pour six nœuds Kafka. Notre poste de test donnait à Docker 4 processeurs et 7,7 Go ; chaque nœud y était limité à 512 Mo (`KAFKA_HEAP_OPTS`), un réglage de laboratoire et non de production.
- Sous Git Bash (Windows), tapez d'abord `export MSYS_NO_PATHCONV=1` dans le terminal hôte : sinon Git Bash transforme les chemins comme `/opt/kafka/...` en chemins Windows.

Les six nœuds

Service et nom réseau	Conteneur	node.id	Rôle	Zone (`broker.rack`)
controller-1	froid-controller-1	1	contrôleur	—
controller-2	froid-controller-2	2	contrôleur	—
controller-3	froid-controller-3	3	contrôleur	—
kafka-1	froid-kafka-1	4	broker	A
kafka-2	froid-kafka-2	5	broker	B
kafka-3	froid-kafka-3	6	broker	C

> **Pourquoi des conteneurs « froid- » ?**
> Un conteneur arrêté garde son nom : tant que les conteneurs `kafka-1` à `kafka-3` des exercices existent, Docker refuse d'en créer d'autres du même nom (« Conflict. The container name "/kafka-1" is already in use »). Le préfixe `froid-` évite ce conflit sans toucher au cluster des exercices. Les noms réseau ne changent pas : les outils Kafka joignent toujours `kafka-1:19092`.

Conventions

- **Terminal hôte** : ouvert dans le dossier `projet-froid-kafka`, il sert aux commandes `docker`.
- **Terminal Kafka** : un shell dans le conteneur `froid-kafka-1`, ouvert avec `docker exec -it froid-kafka-1 bash` puis `cd /opt/kafka`. Il sert aux outils Kafka, avec l'adresse `kafka-1:19092`. Pour le quitter : `exit`.
- Ne tapez pas le `$` du début. Une commande qui continue après un `\` se tape ou se colle telle quelle.
- Les blocs à bord doré montrent une sortie obtenue pendant nos tests du 21/10/2026. Vos identifiants, horodatages et numéros de leader peuvent être différents.
- Chaque phase se termine par un **point de contrôle** : ne passez à la suivante que s'il est atteint.
- La durée indiquée en tête de chaque phase est une estimation, calculée d'après le travail demandé et les attentes mesurées pendant nos tests : votre groupe peut aller plus vite ou plus lentement.

> **Laboratoire, pas production**
> PLAINTEXT n'est admis que sur un réseau de laboratoire isolé ; en production, prévoyez TLS ou SASL et des contrôles d'accès. Ne lancez jamais `docker compose down -v` sur un environnement dont vous voulez garder les données.

Phase A — Cadrer et dimensionner

Durée estimée 1 h 05 **Cours** 2.3, 2.5.2 ; modèle de livrable p. 55 **Déjà vu** atelier de la séance 5, cas 4

1. **Fiche de besoin.** Décrivez producteurs, consommateurs, débit moyen et pic, rétention, pertes et doublons acceptables, exigence d'ordre par clé, et l'intérêt d'un traitement en temps réel plutôt que par lots (chapitre 1).
2. **Calculs.** Le nombre de messages par seconde, en moyenne et en pointe ; le débit entrant en octets/s et en Mo/s ; les données brutes gardées pendant 7 jours ; le stockage total avec `replication.factor=3` ; une capacité cible avec 30 % de marge (hypothèse du TP). Formule simplifiée du cours (p. 53) : stockage brut = débit × rétention × facteur de réplication.
3. **Écarts.** Comparez cette estimation à la capacité de votre poste. Relevez ce que la formule ignore : compression, index, segments, fichiers internes, marge d'exploitation. Le laboratoire n'a pas à stocker sept jours de trafic : vous utiliserez quelques messages de test.

Questions

1. Pourquoi dimensionne-t-on les performances sur le pic, mais le stockage sur le débit moyen ?
2. Quelles hypothèses pourraient rendre vos estimations fausses ?
3. Pourquoi 500 capteurs ne signifient-ils pas 500 partitions ? → 2.2.2, 2.2.6

> **Point de contrôle A**
> Un tableau de calculs dont chaque ligne cite son hypothèse. Vérifiez la cohérence : le stockage avec réplication vaut trois fois le stockage brut, et la marge s'applique au total.

Phase B — Concevoir l'architecture

Durée estimée 45 min **Cours** 2.1.3, 2.5.2, figure 12 (p. 46)

1. **Topologie.** Dessinez les trois contrôleurs (1, 2, 3), les trois brokers (4, 5, 6), les producteurs « capteurs », deux applications consommatrices et le futur job Flink.
2. **Justifications.** `process.roles=controller` ou `broker` selon le nœud ; un `node.id` unique dans tout le cluster ; un quorum majoritaire de trois contrôleurs ; une zone A, B ou C par broker avec `broker.rack` ; les listeners et adresses annoncées ; le nombre de partitions des mesures et des flux de sortie.
3. **Production.** Décrivez ce qui doit changer : trois hôtes ou zones réellement séparés, stockage persistant, disques dédiés, TLS/SASL, réseau, supervision, ressources, sauvegardes et procédures d'exploitation.

> **Point de contrôle B**
> Un schéma annoté : noms, identifiants, ports (contrôleurs 9093, brokers 19092), rôles et zones, conforme au tableau « Les six nœuds ».

Phase C — Construire les six nœuds

Durée estimée 2 h 20 **Cours** 2.5.3 (p. 46-50), figure 13 **Déjà vu** exercice 1

Le fichier `docker-compose-3-brokers.yml` des exercices laissait l'image configurer et formater les nœuds. Ici, vous écrivez vous-mêmes la configuration et vous suivez la séquence du cours : identifiants, fichiers, formatage des contrôleurs, puis des brokers.

C1. Préparer le dossier du projet

ARBORESCENCE

```
projet-froid-kafka/
  .env                       identifiants du cluster (C3)
  docker-compose-froid.yml   les six services (C5)
  config/                    controller-1.properties … controller-3.properties
                             broker-4.properties … broker-6.properties
  preuves/                   sorties de commandes, copies d'écran
  README.md                  démarrage, arrêt, identifiants, choix
```

C2. Vérifier Docker et l'image

TERMINAL HÔTE · DOSSIER PROJET-FROID-KAFKA

```
$ docker --version
$ docker compose version
$ docker pull apache/kafka:4.3.1
```

Notez les versions de Docker et de l'image : elles vont dans le dossier d'architecture.

C3. Générer les identifiants

Il faut un identifiant de cluster et un UUID de répertoire par contrôleur (cours, p. 47). Lancez quatre fois :

TERMINAL HÔTE · DOSSIER PROJET-FROID-KAFKA

```
$ docker run --rm apache/kafka:4.3.1 /opt/kafka/bin/kafka-storage.sh random-uuid
```

```
b_HG7JfaQm2r33EuqqIZXQ
```

Rangez les quatre valeurs dans le fichier `.env` ; Docker Compose le lit tout seul. Ne les changez plus : le premier formatage les écrit dans les volumes.

FICHIER .ENV (VOS QUATRE VALEURS)

```
CLUSTER_ID=b_HG7JfaQm2r33EuqqIZXQ
UUID_C1=6SXWmfpOQEudlaPnRGZ9ug
UUID_C2=br5mPJa6R6C0yzJgyC8X7A
UUID_C3=Xd7FMlzlRzCuTT55RAi7fA
```

C4. Écrire les six fichiers de configuration

Voici le modèle d'un contrôleur et celui d'un broker, tirés du cours (p. 48) et testés. Écrivez les quatre autres fichiers en changeant ce qu'indique le tableau.

FICHIER CONFIG/CONTROLLER-1.PROPERTIES

```
process.roles=controller
node.id=1
listeners=CONTROLLER://controller-1:9093
controller.listener.names=CONTROLLER
listener.security.protocol.map=CONTROLLER:PLAINTEXT
controller.quorum.bootstrap.servers=controller-1:9093,controller-2:9093,controller-3:9093
log.dirs=/var/lib/kafka/data
num.partitions=6
default.replication.factor=3
min.insync.replicas=2
```

FICHIER CONFIG/BROKER-4.PROPERTIES

```
process.roles=broker
node.id=4
broker.rack=A
listeners=PLAINTEXT://kafka-1:19092
advertised.listeners=PLAINTEXT://kafka-1:19092
inter.broker.listener.name=PLAINTEXT
controller.listener.names=CONTROLLER
listener.security.protocol.map=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
controller.quorum.bootstrap.servers=controller-1:9093,controller-2:9093,controller-3:9093
log.dirs=/var/lib/kafka/data
offsets.topic.replication.factor=3
transaction.state.log.replication.factor=3
transaction.state.log.min.isr=2
unclean.leader.election.enable=false
log.retention.hours=168
```

Fichier	Ce qui change
`controller-2`, `controller-3`	`node.id` et l'hôte de `listeners` (`controller-2`, `controller-3`)
`broker-5`, `broker-6`	`node.id` (5, 6), `broker.rack` (B, C), l'hôte de `listeners` et de `advertised.listeners` (`kafka-2`, `kafka-3`)

Un écart avec le modèle du cours

Le cours (p. 48) place aussi `num.partitions` et `default.replication.factor` dans le fichier du broker. Kafka 4.3.1 l'accepte, mais avertit au démarrage : « default.replication.factor is defined in the broker role. This configuration will be ignored in 5.0. Please set default.replication.factor in the controller role instead. » Nous les gardons donc dans les fichiers des contrôleurs, comme `min.insync.replicas` (cours, p. 49).

C5. Écrire le fichier Compose

Voici deux des six services, un contrôleur et un broker, avec la fin du fichier. Chaque service formate son volume puis démarre Kafka ; les commandes longues sont écrites sur plusieurs lignes après `>-`, que YAML rejoint en une seule. L'option `--ignore-formatted` rend le formatage sans effet lors des redémarrages suivants. Les brokers attendent que les trois contrôleurs soient en bonne santé, puis se formatent avec `--no-initial-controllers` (cours, p. 49).

FICHIER DOCKER-COMPOSE-FROID.YML (EXTRAIT)

```
name: projet-froid

services:
  controller-1:
    image: apache/kafka:4.3.1
    container_name: froid-controller-1
    hostname: controller-1
    environment:
      KAFKA_HEAP_OPTS: "-Xmx512m -Xms512m"
      CLUSTER_ID: ${CLUSTER_ID}
      QUORUM_INITIAL: "1@controller-1:9093:${UUID_C1},2@controller-2:9093:${UUID_C2},3@controller-3:9093:${UUID_C3}"
    volumes:
      - ./config:/opt/kafka/config/froid:ro
      - controller-1:/var/lib/kafka/data
    command:
      - bash
      - -c
      - >-
        /opt/kafka/bin/kafka-storage.sh format --ignore-formatted
        --cluster-id $$CLUSTER_ID --initial-controllers $$QUORUM_INITIAL
        --config /opt/kafka/config/froid/controller-1.properties
        && exec /opt/kafka/bin/kafka-server-start.sh /opt/kafka/config/froid/controller-1.properties
    healthcheck:
      test:
        - CMD-SHELL
        - >-
          /opt/kafka/bin/kafka-metadata-quorum.sh --bootstrap-controller $$(hostname):9093
          describe --status | grep -Eq 'LeaderId:[[:space:]]+[0-9]'
      interval: 10s
      timeout: 30s
      retries: 30
      start_period: 20s

  kafka-1:
    image: apache/kafka:4.3.1
    container_name: froid-kafka-1
    hostname: kafka-1
    environment:
      KAFKA_HEAP_OPTS: "-Xmx512m -Xms512m"
      CLUSTER_ID: ${CLUSTER_ID}
    volumes:
      - ./config:/opt/kafka/config/froid:ro
      - kafka-1:/var/lib/kafka/data
    command:
      - bash
      - -c
      - >-
        /opt/kafka/bin/kafka-storage.sh format --ignore-formatted
        --cluster-id $$CLUSTER_ID --no-initial-controllers
        --config /opt/kafka/config/froid/broker-4.properties
        && exec /opt/kafka/bin/kafka-server-start.sh /opt/kafka/config/froid/broker-4.properties
    depends_on:
      controller-1: { condition: service_healthy }
      controller-2: { condition: service_healthy }
      controller-3: { condition: service_healthy }
    healthcheck:
      test:
        - CMD-SHELL
        - >-
          /opt/kafka/bin/kafka-broker-api-versions.sh
          --bootstrap-server $$(hostname):19092 > /dev/null
      interval: 10s
      timeout: 30s
      retries: 30
      start_period: 20s

volumes:
  controller-1:
  controller-2:
  controller-3:
  kafka-1:
  kafka-2:
  kafka-3:
```

Ajoutez `controller-2`, `controller-3`, `kafka-2` et `kafka-3` sur le même modèle. Le double `$$` transmet un `$` au shell du conteneur ; `${CLUSTER_ID}`, lui, est remplacé par Compose à partir du fichier `.env`.

C6. Démarrer le cluster et vérifier le quorum

L'option `--wait` rend la main quand les six services sont en bonne santé : un peu moins de 4 minutes au premier démarrage sur notre poste.

TERMINAL HÔTE · DOSSIER PROJET-FROID-KAFKA

```
$ docker compose -f docker-compose-froid.yml config --quiet
$ docker compose -f docker-compose-froid.yml up -d --wait
$ docker compose -f docker-compose-froid.yml ps --format "{{.Name}} · {{.Status}}"
$ docker logs froid-controller-1 --tail 30
$ docker logs froid-kafka-1 --tail 30
```

```
froid-controller-1 · Up 3 minutes (healthy)
...
froid-kafka-3 · Up About a minute (healthy)
```

Au premier démarrage

Le journal d'un contrôleur doit montrer « Formatting dynamic metadata voter directory /var/lib/kafka/data » ; celui d'un broker, « Formatting metadata directory ». Sur un seul poste, nous avons aussi vu des lignes ERROR (« Direct-to-controller communication is not supported with the current MetadataVersion ») et, au démarrage des brokers, un changement de leader du quorum (du contrôleur 3 au contrôleur 1). Seul compte l'état final ci-dessous.

TERMINAL HÔTE · DOSSIER PROJET-FROID-KAFKA

```
$ docker exec -it froid-kafka-1 bash
```

TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1

```
$ cd /opt/kafka
$ bin/kafka-metadata-quorum.sh --bootstrap-server kafka-1:19092 describe --status
$ bin/kafka-metadata-quorum.sh --bootstrap-server kafka-1:19092 describe --replication
```

```
ClusterId:              b_HG7JfaQm2r33EuqqIZXQ
LeaderId:               1
LeaderEpoch:            9
...
CurrentVoters:          [{"id": 1, "directoryId": "6SXWmfpOQEudlaPnRGZ9ug", ...}, {"id": 2, ...}, {"id": 3,
...}]
CurrentObservers:       [{"id": 4, ...}, {"id": 5, ...}, {"id": 6, ...}]

NodeId  DirectoryId              LogEndOffset  Lag  ...  Status
1       6SXWmfpOQEudlaPnRGZ9ug   325           0    ...  Leader
2       br5mPJa6R6C0yzJgyC8X7A   325           0    ...  Follower
3       Xd7FMlzlRzCuTT55RAi7fA   325           0    ...  Follower
4       4R2vtddiKGDtYT3_tzlgLA   325           0    ...  Observer
5       B4qnQA7t_Tf5jR3GCaa3zQ   325           0    ...  Observer
6       maXbSMoVU04NV6Q2kn1NEg   325           0    ...  Observer
```

Questions

1. Retrouvez vos trois UUID de répertoire dans `CurrentVoters`. Pourquoi les brokers sont-ils des observateurs et non des votants ? → 2.1.3, exercice 1
2. Combien de contrôleurs peuvent tomber sans perte de majorité ? → 2.5.2
3. Que se passerait-il si un broker annonçait dans `advertised.listeners` une adresse que les clients ne peuvent pas joindre ? → 2.5.4

> **Point de contrôle C**
> Six services « healthy » ; trois votants (1, 2, 3) et trois observateurs (4, 5, 6) ; un `Lag` nul ou très faible (nous avons relevé 0, puis 1 lors d'un second essai : la réplication est continue). Si ce n'est pas le cas, corrigez avant de passer à la phase D.

Phase D — Créer les topics métier

Durée estimée 30 min **Cours** 2.2.1, 2.3.3, 2.3.5, 2.3.6 **Déjà vu** exercice 2, atelier de la séance 5

Voici une base de quatre topics, à justifier ou à adapter. Pour chaque topic, fixez `min.insync.replicas=2`. Pour les producteurs critiques, exigez `acks=all` et l'idempotence : ce sont les valeurs par défaut du client Java en 4.3 (cours, p. 17), mais écrivez-les dans leur configuration. Ne promettez pas « zéro perte en toutes circonstances » : précisez la panne couverte.

Topic	Usage	Partitions	RF	Politique
`mesures-brutes`	événements des capteurs	6	3	delete, 7 jours
`mesures-agregees`	résultats futurs de Flink	3	3	delete, 30 jours
`alertes-temperature`	notifications métier	3	3	delete, 30 jours
`etat-capteurs`	dernier état par capteur	3	3	compact

TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1

```
$ bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --create --topic mesures-brutes \
    --partitions 6 --replication-factor 3 --config min.insync.replicas=2 \
    --config cleanup.policy=delete --config retention.ms=604800000
$ bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --describe --topic mesures-brutes
```

```
Topic: mesures-brutes  ...  PartitionCount: 6  ReplicationFactor: 3  Configs:
min.insync.replicas=2,cleanup.policy=delete,retention.ms=604800000,...
  Partition: 0  Leader: 5  Replicas: 5,6,4  Isr: 5,6,4  Elr:   LastKnownElr:
  Partition: 1  Leader: 6  Replicas: 6,4,5  Isr: 6,4,5  Elr:   LastKnownElr:
  Partition: 2  Leader: 4  Replicas: 4,5,6  Isr: 4,5,6  Elr:   LastKnownElr:
  Partition: 3  Leader: 4  Replicas: 4,5,6  Isr: 4,5,6  Elr:   LastKnownElr:
  Partition: 4  Leader: 5  Replicas: 5,6,4  Isr: 5,6,4  Elr:   LastKnownElr:
  Partition: 5  Leader: 6  Replicas: 6,4,5  Isr: 6,4,5  Elr:   LastKnownElr:
```

Créez les trois autres topics en adaptant les options : 30 jours valent `2592000000` ms ; la compaction s'obtient avec `cleanup.policy=compact`, pas avec `retention.ms`. Relevez ensuite la configuration :

TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1

```
$ bin/kafka-configs.sh --bootstrap-server kafka-1:19092 --describe --entity-type topics \
    --entity-name mesures-brutes
```

```
  min.insync.replicas=2 sensitive=false synonyms={DYNAMIC_TOPIC_CONFIG:min.insync.replicas=2,
    DYNAMIC_DEFAULT_BROKER_CONFIG:min.insync.replicas=2, DEFAULT_CONFIG:min.insync.replicas=1}
```

Travaux

1. Remplissez le tableau des leaders, répliques et ISR de chaque partition. Les brokers se partagent-ils le leadership ?
2. Calculez le nombre total de répliques des quatre topics.
3. Dans la sortie de `kafka-configs.sh`, d'où vient la ligne `DYNAMIC_DEFAULT_BROKER_CONFIG:min.insync.replicas=2` ? → 2.3.4, cours p. 49

Point de contrôle D

Quatre topics, chacun avec `ReplicationFactor: 3` et `min.insync.replicas=2`. Dans `mesures-brutes`, chaque broker est leader de deux partitions.

Phase E — Envoyer des mesures de test

Durée estimée 25 min **Cours** 2.2.2, 2.2.4 **Déjà vu** exercice 2

Les capteurs sont simulés par le producteur console. Ouvrez deux terminaux Kafka : le producteur dans le premier, le consommateur dans le second.

TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1

```
$ bin/kafka-console-producer.sh --bootstrap-server kafka-1:19092 --topic mesures-brutes \
    --reader-property parse.key=true --reader-property key.separator=: \
    --command-property acks=all --command-property enable.idempotence=true
```

LIGNES À TAPER DANS LE PRODUCTEUR

```
CAP-001:{"eventId":"E001","sensorId":"CAP-001","ts":"2026-10-21T09:00:00Z","temperatureC":4.2}
CAP-002:{"eventId":"E002","sensorId":"CAP-002","ts":"2026-10-21T09:00:01Z","temperatureC":5.1}
CAP-001:{"eventId":"E003","sensorId":"CAP-001","ts":"2026-10-21T09:00:02Z","temperatureC":8.7}
CAP-001:{"eventId":"E004","sensorId":"CAP-001","ts":"2026-10-21T09:02:05Z","temperatureC":9.0}
CAP-003:{"eventId":"E005","sensorId":"CAP-003","ts":"2026-10-21T09:02:06Z","temperatureC":3.8}
```

TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1

```
$ bin/kafka-console-consumer.sh --bootstrap-server kafka-1:19092 --topic mesures-brutes \
    --from-beginning --formatter-property print.key=true \
    --formatter-property print.partition=true --formatter-property print.offset=true
```

```
Partition:1  Offset:0  CAP-001  {"eventId":"E001","sensorId":"CAP-001",...,"temperatureC":4.2}
Partition:1  Offset:1  CAP-001  {"eventId":"E003","sensorId":"CAP-001",...,"temperatureC":8.7}
Partition:1  Offset:2  CAP-001  {"eventId":"E004","sensorId":"CAP-001",...,"temperatureC":9.0}
Partition:2  Offset:0  CAP-003  {"eventId":"E005","sensorId":"CAP-003",...,"temperatureC":3.8}
Partition:3  Offset:0  CAP-002  {"eventId":"E002","sensorId":"CAP-002",...,"temperatureC":5.1}
```

Deux remarques

L'énoncé d'origine utilisait `--producer-property` : Kafka 4.3.1 répond « --producer-property is deprecated and will be removed in a future version. Use --command-property instead. » Par ailleurs, lors de notre test, les deux premiers essais de lecture n'ont rien affiché : le topic interne `__consumer_offsets` venait d'être créé à cette occasion. Si rien ne s'affiche, patientez, puis relancez le consommateur. Arrêtez-le avec Ctrl-C.

Questions

1. Les messages de CAP-001 restent-ils sur la même partition, et dans leur ordre ? Envoyez d'autres mesures avec la même clé pour le vérifier. → 2.2.2
2. Les offsets sont-ils globaux au topic ? La clé sert-elle seulement à afficher le capteur ? → 2.2.2, 2.2.4
3. Un producteur `acks=all` peut-il créer des doublons métier si l'application envoie volontairement deux fois le même événement ? → 2.3.7

Point de contrôle E

Les trois mesures de CAP-001 sont dans une seule partition, aux offsets 0, 1 et 2.

Phase F — Consommateurs, groupes et relecture

Durée estimée 25 min **Cours** 2.2.5, 2.2.6 **Déjà vu** TP de la séance 4

1. Dans deux terminaux Kafka, lancez un consommateur du groupe `supervision-froid` et un du groupe `analyse-qualite` :

   TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1
   ```
   $ bin/kafka-console-consumer.sh --bootstrap-server kafka-1:19092 --topic mesures-brutes \
       --group supervision-froid --from-beginning --formatter-property print.key=true
   ```

   Répétez avec `--group analyse-qualite`. Chaque groupe reçoit les 5 mesures.

2. Arrêtez le consommateur `analyse-qualite` (Ctrl-C), produisez trois nouvelles mesures, puis décrivez son groupe :

   TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1
   ```
   $ bin/kafka-consumer-groups.sh --bootstrap-server kafka-1:19092 --describe \
       --group analyse-qualite
   ```

   ```
   Consumer group 'analyse-qualite' has no active members.
   GROUP            TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
   analyse-qualite  mesures-brutes  1          3               4               1
   analyse-qualite  mesures-brutes  2          1               2               1
   analyse-qualite  mesures-brutes  3          1               2               1
   ...
   ```

3. Relancez ce consommateur sans `--from-beginning` : il reprend exactement les trois nouvelles mesures.

4. Lancez enfin un nouveau groupe, par exemple `rejeu-incident`, avec `--from-beginning` : il relit tout l'historique encore conservé.

Travaux

1. Expliquez la différence entre deux groupes distincts et deux consommateurs du même groupe. → 2.2.6
2. Que mesurent `CURRENT-OFFSET`, `LOG-END-OFFSET` et `LAG` ? Où Kafka garde-t-il `CURRENT-OFFSET` ? → 2.2.5
3. Pourquoi les mesures relues par le nouveau groupe ne sortent-elles pas dans l'ordre global d'envoi ? → 2.2.2

> **Point de contrôle F**
> Un `LAG` total de 3 pour le groupe arrêté, ramené à 0 après sa relance ; un nouveau groupe qui relit toutes les mesures.

Phase G — Tester la tolérance aux pannes

Durée estimée 55 min **Cours** 2.3.2 à 2.3.4, figure 8 **Déjà vu** exercice 4, TP de la séance 6

Les contrôleurs étant séparés des brokers, le quorum reste disponible pendant tout ce scénario.

G1. Situation initiale

TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1
```
$ bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --describe --topic mesures-brutes
```

Notez `Leader`, `Replicas` et `Isr` de chaque partition. Les trois brokers doivent être dans l'ISR.

G2. Perte d'un broker

TERMINAL HÔTE · DOSSIER PROJET-FROID-KAFKA
```
$ docker stop froid-kafka-3
```

TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1

```
$ bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --describe --topic mesures-brutes
$ bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --describe --under-replicated-partitions
```

```
  Partition: 0  Leader: 5  Replicas: 5,6,4  Isr: 5,4  ...
  Partition: 1  Leader: 4  Replicas: 6,4,5  Isr: 4,5  ...
  ...
  Partition: 5  Leader: 4  Replicas: 6,4,5  Isr: 4,5  ...
```

Lors de notre test, 65 partitions étaient sous-répliquées : les 50 de `__consumer_offsets` et les 15 des topics métier. Produisez une mesure avec `acks=all` : elle est acceptée.

G3. Perte d'un deuxième broker

TERMINAL HÔTE · DOSSIER PROJET-FROID-KAFKA

```
$ docker stop froid-kafka-2
```

Décrivez le topic, puis essayez d'envoyer une mesure avec le producteur de la phase E.

```
  Partition: 0  Leader: 4  Replicas: 5,6,4  Isr: 4  Elr: 5  LastKnownElr:
  ...
WARN [Producer clientId=console-producer] Got error produce response with correlation id 4 on
  topic-partition mesures-brutes-1, retrying (2 attempts left). Error: NOT_ENOUGH_REPLICAS
...
ERROR Error when sending message to topic mesures-brutes with key: 7 bytes, value: 86 bytes with error:
org.apache.kafka.common.errors.NotEnoughReplicasException: Messages are rejected since there are fewer
in-sync replicas than required.
```

Le producteur console abandonne seul

Il est inutile de régler `delivery.timeout.ms` : le producteur console abandonne après 3 nouvelles tentatives, son réglage par défaut (`--message-send-max-retries` , défaut 3 ; `--request-timeout-ms` , défaut 1500). Lors de notre test, l'erreur est arrivée en moins de 10 secondes. Le client Java, lui, réessaie bien plus longtemps (`delivery.timeout.ms` : 2 minutes par défaut, cours p. 17).

G4. Restaurer le cluster

TERMINAL HÔTE · DOSSIER PROJET-FROID-KAFKA

```
$ docker start froid-kafka-2 froid-kafka-3
```

Relancez `--under-replicated-partitions` jusqu'à ce qu'il ne renvoie plus rien : moins d'une minute lors de notre test. Les leaders restent d'abord sur le broker 4 ; ils reviennent ensuite sur les leaders préférés au contrôle suivant (`auto.leader.rebalance.enable=true` , toutes les 300 secondes) : lors de notre test, c'était fait moins de 6 minutes après la relance.

Moment	Leaders	ISR	Écriture acks=all
État initial			
Broker 6 arrêté			
Brokers 6 et 5 arrêtés			
Retour des brokers 5 et 6			
Six minutes plus tard			

Travaux

1. Pourquoi `acks=all` et `min.insync.replicas=2` protègent-ils les nouvelles écritures en les refusant ? → 2.3.3, figure 8
2. Pourquoi une partition sous-répliquée n'est-elle pas toujours indisponible ? → 2.3.3
3. Avec deux brokers arrêtés, la colonne `Elr` affiche 5. Que signifie-t-elle ? Comparez avec vos relevés de l'exercice 2, où elle était vide. → 2.3.4 (p. 30-31)
4. Pourquoi `__consumer_offsets` apparaît-il dans les partitions sous-répliquées ? → 2.2.5

> **Point de contrôle G**
> Un broker arrêté : ISR à deux brokers et écritures acceptées. Deux brokers arrêtés : `NotEnoughReplicasException`. Après la relance : aucune partition sous-répliquée. Ne poursuivez pas tant qu'un topic reste dégradé.

Phase H — Segments, rétention et compaction

Durée estimée 55 min **Cours** 2.2.3, 2.3.5, 2.3.6 **Déjà vu** exercices 3 et 5

H1. Le stockage sur disque

TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1

```
$ bin/kafka-configs.sh --bootstrap-server kafka-1:19092 --describe --all --entity-type brokers \
    --entity-name 4 | grep -E "^ *log\.(dirs|segment\.bytes|roll\.hours)="
$ ls /var/lib/kafka/data
$ ls -l /var/lib/kafka/data/mesures-brutes-1
$ bin/kafka-dump-log.sh --files /var/lib/kafka/data/mesures-brutes-1/00000000000000000000.log \
    --print-data-log
```

```
  log.dirs=/var/lib/kafka/data sensitive=false ...
-rw-r--r-- 1 appuser appuser 10485760 Oct 21 08:42 00000000000000000000.index
-rw-r--r-- 1 appuser appuser      530 Oct 21 08:50 00000000000000000000.log
-rw-r--r-- 1 appuser appuser 10485756 Oct 21 08:42 00000000000000000000.timeindex
baseOffset: 0 lastOffset: 2 count: 3 baseSequence: 0 lastSequence: 2 producerId: 0 ...
| offset: 0 ... keySize: 7 valueSize: 86 sequence: 0 ... key: CAP-001 payload: {"eventId":"E001",...}
| offset: 1 ... keySize: 7 valueSize: 86 sequence: 1 ... key: CAP-001 payload: {"eventId":"E003",...}
| offset: 2 ... keySize: 7 valueSize: 86 sequence: 2 ... key: CAP-001 payload: {"eventId":"E004",...}
```

Choisissez la partition qui contient vos mesures de CAP-001 (phase E).

Questions

1. Pourquoi les trois mesures forment-elles un seul lot (`count: 3`) ? → 2.2.4
2. À quoi servent `producerId` et `sequence` ? → 2.3.7
3. Quel lien voyez-vous entre offset, segment et politique de rétention ? → 2.2.3, 2.3.5

H2. Le topic compacté

Pour rendre la compaction visible en laboratoire seulement, raccourcissez la durée d'un segment et le seuil de nettoyage, puis envoyez cinq valeurs :

TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1

```
$ bin/kafka-configs.sh --bootstrap-server kafka-1:19092 --alter --entity-type topics \
    --entity-name etat-capteurs --add-config segment.ms=20000,min.cleanable.dirty.ratio=0.01
$ bin/kafka-console-producer.sh --bootstrap-server kafka-1:19092 --topic etat-capteurs \
    --reader-property parse.key=true --reader-property key.separator=:
```

LIGNES À TAPER DANS LE PRODUCTEUR

```
CAP-001:4.2
CAP-002:5.1
CAP-001:8.7
CAP-003:3.8
CAP-001:9.0
```

Attendez plus de 20 secondes, puis tapez `CAP-001:9.2`. **Gardez la clé CAP-001** : `etat-capteurs` a trois partitions, et un segment ne se ferme qu'à l'arrivée d'un événement dans sa propre partition. Patientez environ une minute, puis relisez le topic depuis le début, avec partitions et offsets :

```
avant :                              une minute après CAP-001:9.2 :
Partition:1  Offset:0  CAP-001  4.2  Partition:2  Offset:0  CAP-003  3.8
Partition:1  Offset:1  CAP-001  8.7  Partition:1  Offset:2  CAP-001  9.0
Partition:1  Offset:2  CAP-001  9.0  Partition:0  Offset:0  CAP-002  5.1
Partition:2  Offset:0  CAP-003  3.8  Partition:1  Offset:3  CAP-001  9.2
Partition:0  Offset:0  CAP-002  5.1
```

Envoyez ensuite un tombstone pour CAP-002, relisez, puis retirez les réglages de démonstration :

TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1

```
$ bin/kafka-console-producer.sh --bootstrap-server kafka-1:19092 --topic etat-capteurs \
    --reader-property parse.key=true --reader-property key.separator=: \
    --reader-property null.marker=NULL
$ bin/kafka-configs.sh --bootstrap-server kafka-1:19092 --alter --entity-type topics \
    --entity-name etat-capteurs --delete-config segment.ms,min.cleanable.dirty.ratio
```

LIGNES À TAPER DANS LE PRODUCTEUR

```
CAP-002:NULL
```

```
Partition:0  Offset:1  CAP-002  null
```

La compaction est asynchrone : l'instant exact où les anciennes valeurs disparaissent n'est pas garanti.

Questions

1. Quelles valeurs ont disparu ? Les offsets restants ont-ils été renumérotés ? → 2.3.6
2. Pourquoi la valeur 9.0 reste-t-elle, alors que 9.2 est plus récente ? → 2.3.6 (segment actif)
3. Pourquoi `cleanup.policy=compact` ne remplace-t-il pas la conservation de tout l'historique ?

> **Point de contrôle H**
> Un segment lu avec `kafka-dump-log.sh` ; les offsets 0 et 1 de CAP-001 disparus, les autres inchangés ; un tombstone visible ; les réglages de démonstration retirés.

Phase I — Réaffecter des répliques et rétablir les leaders préférés

Durée estimée 35 min **Cours** 2.3.4 (p. 31), 2.5.4 **Déjà vu** exercice 6

Travaillez sur un topic de démonstration, `tp-migration` : ne touchez pas au facteur de réplication d'un topic métier.

1. Créez `tp-migration` avec 3 partitions et une seule réplique, puis décrivez-le.

TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1

```
$ bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --create --topic tp-migration \
    --partitions 3 --replication-factor 1
$ bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --describe --topic tp-migration
```

```
  Partition: 0  Leader: 5  Replicas: 5  Isr: 5 ...
  Partition: 1  Leader: 6  Replicas: 6  Isr: 6 ...
  Partition: 2  Leader: 4  Replicas: 4  Isr: 4 ...
```

2. Écrivez la liste du topic à déplacer, demandez un plan et gardez le bloc « Current partition replica assignment » : il permet de revenir en arrière.

TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1

```
$ echo '{"topics": [{"topic": "tp-migration"}], "version": 1}' > /tmp/topics-migration.json
$ bin/kafka-reassign-partitions.sh --bootstrap-server kafka-1:19092 \
    --topics-to-move-json-file /tmp/topics-migration.json --broker-list 4,5,6 --generate
```

3. Écrivez votre plan : trois répliques par partition, dans les ordres [4,5,6], [5,6,4] et [6,4,5]. Collez le bloc d'un coup, de `cat` à `FIN` ; écrire le fichier dans le conteneur évite les problèmes d'encodage d'un fichier copié depuis Windows.

TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1

```
$ cat > /tmp/plan-migration.json <<'FIN'
{"version": 1, "partitions": [
  {"topic": "tp-migration", "partition": 0, "replicas": [4, 5, 6]},
  {"topic": "tp-migration", "partition": 1, "replicas": [5, 6, 4]},
  {"topic": "tp-migration", "partition": 2, "replicas": [6, 4, 5]}]}
FIN
```

4. Exécutez le plan, vérifiez-le, puis décrivez le topic.

TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1

```
$ bin/kafka-reassign-partitions.sh --bootstrap-server kafka-1:19092 \
    --reassignment-json-file /tmp/plan-migration.json --execute
$ bin/kafka-reassign-partitions.sh --bootstrap-server kafka-1:19092 \
    --reassignment-json-file /tmp/plan-migration.json --verify
$ bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --describe --topic tp-migration
```

```
Successfully started partition reassignments for tp-migration-0,tp-migration-1,tp-migration-2
Reassignment of partition tp-migration-0 is completed.
...
  Partition: 0  Leader: 5  Replicas: 4,5,6  Isr: 4,5,6 ...
  Partition: 1  Leader: 6  Replicas: 5,6,4  Isr: 4,5,6 ...
  Partition: 2  Leader: 4  Replicas: 6,4,5  Isr: 4,5,6 ...
```

Le leader effectif n'est pas le premier broker de la liste, c'est-à-dire le leader préféré.

5. Rendez le leadership aux leaders préférés, pour ce seul topic. Une élection avec `--all-topic-partitions` toucherait tout le cluster ; le fichier JSON limite l'élection aux trois partitions.

TERMINAL KAFKA · CONTENEUR FROID-KAFKA-1

```
$ cat > /tmp/election-migration.json <<'FIN'
{"partitions": [
  {"topic": "tp-migration", "partition": 0},
  {"topic": "tp-migration", "partition": 1},
  {"topic": "tp-migration", "partition": 2}]}
FIN
$ bin/kafka-leader-election.sh --bootstrap-server kafka-1:19092 --election-type PREFERRED \
    --path-to-json-file /tmp/election-migration.json
$ bin/kafka-topics.sh --bootstrap-server kafka-1:19092 --describe --topic tp-migration
```

```
Successfully completed leader election (PREFERRED) for partitions tp-migration-2, tp-migration-1, tp-migration-0
  Partition: 0  Leader: 4  Replicas: 4,5,6  Isr: 4,5,6 ...
  Partition: 1  Leader: 5  Replicas: 5,6,4  Isr: 4,5,6 ...
  Partition: 2  Leader: 6  Replicas: 6,4,5  Isr: 4,5,6 ...
```

Questions

1. Pourquoi garder le plan initial ? À quoi servent `--generate`, `--execute` et `--verify` ? → 2.5.4
2. Pourquoi le premier broker de `Replicas` n'est-il pas leader dès la fin de la réaffectation ? → 2.3.4 (p. 31)
3. Relancez l'élection : que répond Kafka ? Quel réglage aurait rétabli seul les leaders préférés ? → 2.3.4, phase G4

> **Point de contrôle I**
> `ReplicationFactor: 3` pour `tp-migration`, puis des leaders égaux au premier broker de chaque liste. Une seconde élection répond « Valid replica already elected ».

Phase J — Exploitation, sécurité et documentation

Durée estimée 45 min **Cours** 2.5.4 (p. 51-52) ; surveillance détaillée au chapitre 4

Concevez un tableau de bord de supervision et un plan d'intervention. Il n'est pas demandé d'installer Prometheus ou Grafana : documentez les métriques essentielles et leur sens.

Élément à surveiller	Objectif
`UnderReplicatedPartitions`	détecter la sous-réplication
`UnderMinIsrPartitionCount`	repérer les partitions sous leur minimum d'ISR
`OfflinePartitionsCount`	détecter les partitions sans leader disponible
`ActiveControllerCount`	surveiller le contrôleur actif
`records-lag-max`, `LAG`	détecter les consommateurs en retard
Espace disque et journaux	prévenir la saturation, diagnostiquer les erreurs

1. Rédigez une procédure de maintenance progressive d'un broker, appuyée sur l'arrêt contrôlé observé en phase G.
2. Justifiez `unclean.leader.election.enable=false`, `broker.rack`, des sauvegardes adaptées, l'isolation des réseaux et TLS/SASL en production.
3. Proposez une procédure de reprise en cas de panne disque. Expliquez pourquoi la réplication ne remplace pas les sauvegardes, et pourquoi une rétention de sept jours n'est pas un archivage légal.

Phase K — Simuler les sorties, préparer Flink

Durée estimée 10 min **Suite** chapitre 3

Sans Flink, alimentez à la main `mesures-agregees` et `alertes-temperature` avec le producteur console, pour simuler les résultats d'un traitement. Dites clairement dans votre présentation qu'il s'agit d'une simulation, pas d'une détection automatique.

LIGNES À TAPER DANS LE PRODUCTEUR

```
CAP-001:{"alertId":"A001","sensorId":"CAP-001","regle":"T>8 pendant 2 min","source":"SIMULATION"}
```

Extension, au chapitre 3 : un job Flink qui lit `mesures-brutes`, travaille en temps d'événement, écarte les valeurs invalides, agrège par capteur et écrit dans les topics de sortie. Pour l'alerte sur deux minutes, il faudra définir la sémantique des fenêtres et le traitement des données en retard.

1. Quelles opérations relèvent de Kafka, et lesquelles de Flink ? Distinguez le transport fiable des mesures de leur interprétation métier.

Phase L — Clôture et arrêt

Durée estimée 15 min

Avant l'arrêt, vérifiez : quorum présent et trois contrôleurs votants ; trois brokers actifs ; topics métier présents ; un leader et un ISR complet pour chaque partition critique ; groupes capables de relire ; changements de configuration consignés ; preuves enregistrées dans `preuves/`.

TERMINAL HÔTE · DOSSIER PROJET-FROID-KAFKA

```
$ docker compose -f docker-compose-froid.yml stop
$ docker compose -f docker-compose-froid.yml start
```

```
All of the log directories are already formatted.
```

`stop` puis `start` relance le cluster sans reformater : le journal de chaque nœud affiche la ligne ci-dessus. Les données sont dans des volumes nommés : lors de notre test, elles ont survécu à `stop` et même à `down`, qui supprime pourtant les conteneurs. Seul `down -v` supprime les volumes, donc les données : ne le lancez qu'avec l'accord de l'enseignant.

Livrables et évaluation

Durée estimée 3 h 35 : dossier 2 h, projet technique 30 min, préparation de l'oral 1 h, relecture des repères 5 min

Livrables

1. **Dossier d'architecture** (8 à 12 pages), selon le modèle de livrable du cours (p. 55) : contexte et hypothèses ; architecture ; dimensionnement ; configuration ; vérifications ; exploitation ; avantages et limites (Kafka face au traitement par lots, Kafka face à Flink, risques restants).
2. **Projet technique** : `.env`, `docker-compose-froid.yml`, les six fichiers de configuration, `README.md`, les fichiers JSON de la phase I et les preuves. Ne diffusez pas de secret si vous avez ajouté une authentification.
3. **Présentation orale** de 10 minutes : démonstration et justification, pas une lecture des commandes.

Grille d'évaluation (proposition, sur 20)

Critère	Points
Analyse métier et dimensionnement justifié	4
Architecture KRaft et configurations reproductibles	4
Topics, clés, production, groupes et relecture	4
Pannes, ISR, durabilité et restauration démontrées	4
Compaction, réaffectation et leaders préférés	2
Supervision, sécurité, qualité des preuves	2
Total	**20**

Questions à défendre à l'oral

1. Pourquoi trois contrôleurs KRaft et non un seul, alors que le cluster a trois brokers ?
2. En quoi `acks=all` ne suffit-il pas sans `min.insync.replicas` ?
3. Pourquoi une partition peut-elle être sous-répliquée tout en restant lisible et accessible en écriture ?
4. Pourquoi `__consumer_offsets` apparaît-il parfois dans les partitions sous-répliquées ?
5. Une réplication sur trois brokers dispense-t-elle de sauvegardes et de `fsync` dans tous les scénarios de panne ?
6. Pourquoi la clé `sensorId` compte-t-elle pour l'ordre des mesures ?
7. Pourquoi Kafka garde-t-il les messages après leur lecture ?
8. Que perd-on avec la compaction, et que garde-t-on ?
9. À quoi servent `--generate`, `--execute` et `--verify` pendant une réaffectation ?
10. Quels changements faudrait-il pour transformer ce laboratoire en service industriel ?

Pour vous orienter à la fin

Avant la présentation orale : cochez chaque ligne

N°	Critère de recette	Preuve à garder	Ce qu'elle doit montrer
1	Quorum sain	`describe --status` (C6)	trois votants (1, 2, 3), un `LeaderId`
2	Brokers 4, 5 et 6	`describe --replication` (C6)	4, 5 et 6 en `Observer`, `Lag` nul ou très faible
3	Topics répliqués, durabilité justifiée	`--describe` et `kafka-configs.sh` (D) ; calculs (A)	`ReplicationFactor: 3`, `min.insync.replicas=2`, choix justifiés
4	Écriture avec clé, lecture par deux groupes	sortie du consommateur (E) ; `kafka-consumer-groups.sh` (F)	une partition par capteur ; un `LAG` qui revient à 0
5	Sortie et retour dans l'ISR	trois `--describe` (G1, G2, G4)	ISR à trois, puis à deux, puis à trois
6	Refus des écritures	message du producteur (G3)	`NotEnoughReplicasException`
7	Rétention, compaction, réaffectation	`kafka-dump-log.sh`, relectures (H) ; `--verify` et élection (I)	valeurs compactées, offsets inchangés ; leaders préférés rétablis
8	Supervision, sécurité, remise en service	tableau de bord et procédures (J) ; arrêt et relance (L)	métriques expliquées ; « already formatted » au redémarrage

> **Si un critère manque**
>
> Revenez au point de contrôle de la phase indiquée : il dit ce qu'il faut observer avant d'aller plus loin. Les phases A, B et J se prouvent par le dossier d'architecture, les autres par les sorties de commandes rangées dans `preuves/`.

Sources

- TP fourni : « TP fil rouge — Déployer un cluster Kafka de A à Z » (MCCI Business School, Master EADL 2), dont reprennent le scénario FROIDIS, les besoins, les phases, la grille et les questions de soutenance.
- Cours du chapitre 2, V3 : sections 2.1 à 2.5 ; pages citées : 17, 30-31, 46-55.
- Chapitre 2, Exercices V1 (six exercices) ; atelier de la séance 5 V1.
- Kafka 4.3.1, image `apache/kafka:4.3.1` : aide intégrée des outils et configuration effective des nœuds. Toutes les phases C à L ont été réalisées le 21/10/2026 sur un cluster neuf (Docker 28.1.1, Windows 10) ; les sorties affichées viennent de ces tests. Pour automatiser le test, les consommateurs y étaient arrêtés par `--max-messages` au lieu de Ctrl-C.